

1. Основы разработки, алгоритмы и качество кода

1.1. Базовые понятия программирования

1. Что такое алгоритм, зачем он нужен и как можно описать простой алгоритм на практике?
2. Что такое условные операторы, циклы, функции и методы, для каких задач они применяются?
3. Что такое рекурсия, в каких случаях она используется и какие риски у рекурсивных решений?
4. Чем класс отличается от интерфейса и какую роль они играют в проектировании программы?
5. Что такое клиент-серверное приложение и как обычно происходит взаимодействие клиента и сервера?

1.2. ООП и проектирование

1. Какие основные принципы ООП существуют и зачем они нужны?
2. Что такое инкапсуляция, наследование, полиморфизм и абстракция?
3. Какие принципы SOLID существуют и какую проблему решает каждый из них?
4. Чем SRP, OCP, LSP, ISP и DIP помогают при проектировании поддерживаемого кода?
5. Какие паттерны проектирования используются чаще всего и в каких ситуациях они применяются?
6. Чем Factory Method отличается от Abstract Factory?
7. Где применяются Strategy, Adapter и Decorator?

1.3. Алгоритмическая сложность и структуры данных

1. Что такое Big O и как с его помощью оценивается сложность алгоритма?
2. Какая сложность у поиска в массиве и в хеш-таблице?
3. Что такое стек, очередь, хеш-таблица и дерево?
4. Какие алгоритмы сортировки используются на практике и чем они отличаются?
5. Какие существуют способы обхода дерева?

1.4. Тестирование

1. Что такое тестирование ПО и чем тестирование белого ящика отличается от черного ящика?
2. Что такое пирамида тестирования и зачем она нужна?
3. Чем unit-тесты отличаются от интеграционных тестов?
4. Что такое TDD и в каких случаях этот подход полезен?
5. Для чего нужны mock и stub, чем они отличаются?
6. Что такое нагрузочное тестирование и какие задачи оно решает?

1.5. Качество кода и процессы разработки

1. Чем Waterfall отличается от Agile?
2. Что такое Scrum и Kanban, в чем разница между этими подходами?

3. Что такое статический анализ кода и какие проблемы он помогает находить?
4. Для чего используются SonarQube, Checkstyle, SpotBugs и PMD?

2. SQL и реляционные базы данных

2.1. Реляционная модель и связи

1. Что такое реляционная база данных и как в ней организовано хранение данных?
2. Что такое таблица, строка, колонка, primary key и foreign key?
3. Как связываются таблицы между собой?
4. Какие типы связей бывают в БД: one-to-one, one-to-many, many-to-many?
5. Чем простой ключ отличается от составного ключа?

2.2. Нормализация и структура данных

1. Что такое нормализация и зачем она применяется?
2. Какие нормальные формы существуют и какие проблемы они устраняют?
3. Когда нормализация полезна, а когда может потребоваться денормализация?

2.3. SQL-запросы и JOIN

1. Чем `WHERE` отличается от `HAVING` ?
2. Какие виды `JOIN` существуют и чем `INNER JOIN` отличается от `LEFT JOIN` ?
3. Для чего используется `GROUP BY` ?
4. Как строится запрос с фильтрацией, группировкой и соединением таблиц?

2.4. Индексы и оптимизация

1. Что такое индекс в БД и как он ускоряет поиск?
2. Какие типы индексов используются в реляционных БД?
3. Какие минусы есть у индексов?
4. Что такое план выполнения запроса и как его посмотреть?
5. Как анализировать и оптимизировать медленный SQL-запрос?

2.5. Транзакции и ACID

1. Что такое транзакция и какие гарантии дает ACID?
2. Что означают Atomicity, Consistency, Isolation и Durability?
3. Какие уровни изоляции транзакций существуют?
4. Что такое dirty read, non-repeatable read, phantom read и lost update?
5. Как уровень изоляции влияет на согласованность данных и производительность?

2.6. Масштабирование и отказоустойчивость

1. Что такое горизонтальное масштабирование?

2. Чем шардирование отличается от партиционирования?
3. Какие задачи решает партиционирование и как выбираются таблицы для партиционирования?
4. Что такое репликация и чем master-slave отличается от multi-master?
5. Как обычно распределяются операции чтения и записи при репликации?

2.7. OLTP, OLAP и распределенные транзакции

1. Что такое OLTP и OLAP, чем они отличаются?
2. Что такое распределенная транзакция и для чего она нужна?
3. Как работает two-phase commit?
4. Какие проблемы есть у распределенных транзакций?

2.8. WAL и журнал транзакций

1. Что такое WAL и зачем нужен transaction log?
2. Как WAL помогает восстанавливать данные после сбоя?
3. Почему WAL важен для durability?

3. Java Core

3.1. Платформа Java

1. Что такое JDK, JRE и JVM, чем они отличаются?
2. Что входит в JDK и что нужно установить для запуска Java-приложения?
3. Как скомпилировать и запустить Java-приложение из консоли?
4. Какие системы сборки Java-проектов используются?
5. Чем Maven отличается от Gradle?

3.2. Типы данных, модификаторы и базовый синтаксис

1. Какие типы данных есть в Java?
2. Чем примитивные типы отличаются от ссылочных?
3. Какие модификаторы доступа есть в Java?
4. Чем отличаются `public`, `private`, `protected` и `package-private`?

3.3. Коллекции и HashMap

1. Для чего нужны коллекции и как устроена их общая иерархия?
2. Чем отличаются `List`, `Set`, `Queue` и `Map`?
3. Как устроен `HashMap` внутри?
4. Что такое коллизия в `HashMap` и как она обрабатывается?
5. Какой контракт существует между `equals` и `hashCode`?
6. Что может произойти при нарушении контракта `equals/hashCode`?

3.4. Исключения

1. Как работает конструкция `try-catch-finally` ?
2. Выполняется ли `finally`, если в блоке `try` есть `return` ?
3. Чем checked exceptions отличаются от unchecked exceptions?
4. Чем `Error` отличается от `Exception` ?

3.5. Generics

1. Что такое generics и зачем они нужны?
2. Чем generic class отличается от generic method?
3. Что такое инвариантность generics?
4. Почему `List<String>` нельзя присвоить в `List<Object>` ?
5. Что такое PECS?
6. Чем отличаются `? extends T` и `? super T` ?
7. Что такое стирание типов и какие ограничения оно накладывает?

3.6. Lambda, functional interface и Stream API

1. Что такое лямбда-выражение и где оно применяется?
2. Что такое функциональный интерфейс?
3. Для чего нужна аннотация `@FunctionalInterface` ?
4. Как написать метод, принимающий лямбду?
5. Что такое Stream API и чем stream отличается от коллекции?
6. Какие основные операции Stream API используются?
7. Чем `map` отличается от `flatMap` ?
8. Чем `filter` отличается от `peek` ?

3.7. Reflection, annotations и proxy

1. Что такое reflection и какие возможности она дает?
2. Где reflection применяется на практике?
3. Как reflection работает с аннотациями?
4. Что такое динамический прокси?
5. Какие способы создания динамических прокси существуют?
6. Чем JDK Dynamic Proxy отличается от CGLIB и ByteBuddy?

3.8. Современные возможности Java

1. Какие важные нововведения появились в последних LTS-версиях Java?
2. Что такое records и для каких задач они подходят?
3. Что такое sealed classes и какие ограничения они задают?
4. Что такое pattern matching и как он упрощает код?
5. Что такое virtual threads и какие задачи они решают?

4. JVM, многопоточность и производительность

4.1. Память JVM

1. Какие области памяти есть в JVM?
2. Что хранится в heap?
3. Что хранится в stack?
4. Что означают параметры JVM `-Xms` , `-Xmx` , `-Xss` ?

4.2. Class Loading

1. Что такое class loading?
2. Какие загрузчики классов есть в Java?
3. Как происходит загрузка класса в JVM?

4.3. Поток и конкурентность

1. Чем процесс отличается от потока?
2. Какие способы создания потоков есть в Java?
3. Чем `Thread` отличается от `Runnable` ?
4. Чем `Runnable` отличается от `Callable` ?
5. Что такое синхронизация в многопоточности?
6. Для чего нужен `synchronized` ?
7. Что такое `volatile` ?
8. Что такое happens-before?
9. Чем `AtomicInteger` отличается от обычного `int` ?
10. Что такое race condition?
11. Что такое deadlock, как он возникает и как его избежать?

4.4. Garbage Collector

1. Что такое Garbage Collector и какую задачу он решает?
2. Как работает GC в Java на базовом уровне?
3. Что такое young generation и old generation?
4. Чем minor GC отличается от full GC?
5. Когда требуется настройка GC?
6. Какие сборщики мусора существуют?
7. Чем G1 отличается от ZGC и Shenandoah?
8. Что такое GC logs, как их включить и что можно по ним понять?

4.5. Профилирование и анализ памяти

1. Что такое profiler и зачем он нужен?
2. Какие Java-профайлеры используются на практике?
3. Для чего применяются VisualVM, JFR, async-profiler, YourKit и JProfiler?

4. Как профайлер может влиять на приложение и почему результаты замеров могут искажаться?
5. Что такое memory dump и heap dump?
6. Как по heap dump анализировать memory leak?

5. Spring и Spring Boot

5.1. Основы Spring

1. Что такое Spring и какие проблемы он решает?
2. Чем Spring Boot отличается от классического Spring?
3. Что нужно для запуска Spring Boot-приложения?
4. Что делает `@SpringBootApplication`?
5. Какие основные Spring-аннотации используются?
6. Чем отличаются `@Component`, `@Service`, `@Repository` и `@Controller`?

5.2. IoC, DI и контейнер

1. Что такое IoC и DI?
2. Какие виды внедрения зависимостей существуют в Spring?
3. Что такое bean?
4. Что такое ApplicationContext?
5. Чем BeanFactory отличается от ApplicationContext?
6. Что такое жизненный цикл Spring bean?
7. Какие bean scopes существуют?
8. Чем singleton scope отличается от prototype scope?
9. Какие web scopes есть в Spring?

5.3. BeanPostProcessor и BeanFactoryPostProcessor

1. Что такое BeanPostProcessor?
2. Что такое BeanFactoryPostProcessor?
3. Чем BeanPostProcessor отличается от BeanFactoryPostProcessor?
4. Где Spring использует BeanPostProcessor на практике?

5.4. Spring Boot starters и автоконфигурация

1. Что такое Spring Boot starter?
2. Что такое автоконфигурация?
3. Как работают `@ConditionalOnClass`, `@ConditionalOnMissingBean` и `@ConditionalOnProperty`?
4. Как реализовать собственный Spring Boot starter?

5.5. Spring MVC и WebFlux

1. Что такое Spring Web MVC?
2. Что такое DispatcherServlet?

3. Как HTTP-запрос попадает в controller?
4. Что такое Spring WebFlux?
5. Чем Spring MVC отличается от WebFlux?
6. Что такое `Flux` и `Mono`?
7. Как работают router и handler в WebFlux?
8. Что такое WebClient?

5.6. Spring Data

1. Что такое Spring Data?
2. Как работает Spring Data Repository?
3. Чем `CrudRepository` отличается от `JpaRepository`?

5.7. AOP, проху и аннотации поведения

1. Как Spring создает прокси для `@Transactional`, `@Cacheable` и `@Async`?
2. Чем JDK проху отличается от CGLIB проху в Spring?
3. Как работает `@Transactional` и какие ограничения у этой аннотации?
4. Как работает `@Cacheable` и какие ограничения у кэширования через Spring?
5. Как работает `@Async`?
6. Почему self-invocation ломает `@Transactional`, `@Cacheable` и `@Async`?
7. Как самостоятельно реализовать упрощенный аналог `@Transactional`?
8. Как самостоятельно реализовать упрощенный аналог `@Cacheable`?
9. Как самостоятельно реализовать упрощенный аналог `@Async`?

5.8. Spring Cloud

1. Что такое Spring Cloud?
2. Какие компоненты Spring Cloud используются чаще всего?
3. Для чего нужны OpenFeign, Config Server, Gateway и Discovery-сервис?
4. Чем Eureka/Discovery отличается от балансировки через Gateway или Kubernetes Service?

5.9. Тестирование Spring-приложений

1. Как тестировать Spring-приложение?
2. Чем `@Mock` отличается от `@MockBean`?
3. Чем unit-тест отличается от `@SpringBootTest`?
4. Для чего используются `@WebMvcTest` и `@DataJpaTest`?

6. MongoDB

6.1. Документная модель

1. К какому классу систем относится MongoDB?
2. Для каких задач применяется MongoDB?

3. В чем преимущество MongoDB перед реляционными БД?
4. Как MongoDB хранит данные?
5. Что такое database, collection и document?
6. Как связаны database, collection и document?
7. Как строятся запросы в MongoDB?

6.2. Индексы

1. Какие виды индексов есть в MongoDB?
2. Как пользоваться индексами в MongoDB?
3. Что такое compound index?
4. Что такое TTL index?

6.3. Репликация и отказоустойчивость

1. Что такое replica set и для чего он нужен?
2. Как MongoDB обеспечивает отказоустойчивость?
3. Что такое oplog?
4. Как oplog участвует в репликации?
5. Как oplog помогает при восстановлении данных?
6. Что происходит, если secondary сильно отстает от primary?

6.4. Read concern, write concern и consistency

1. Что такое read concern?
2. Что такое write concern?
3. Что такое eventual consistency?
4. Как read/write concern влияют на consistency, durability и скорость операций?

6.5. Транзакции и персистентность

1. Как работают транзакции в MongoDB?
2. Какие гарантии дают транзакции?
3. За счет какого механизма MongoDB гарантирует персистентность?
4. Чем journal отличается от oplog?

6.6. Шардирование и CAP

1. Как устроен шардированный кластер MongoDB?
2. Что такое shard, config server и mongos?
3. Что такое shard key и какие проблемы возникают при плохом выборе ключа?
4. Какие гарантии MongoDB дает с точки зрения CAP-теоремы?
5. Как конфигурация MongoDB влияет на consistency и availability?

6.7. WiredTiger

1. Что такое WiredTiger?

2. Какие блокировки использует WiredTiger?
3. Как WiredTiger работает с памятью и кешем?

7. Redis

7.1. Назначение и модель хранения

1. К какому классу систем относится Redis?
2. Для каких задач используется Redis?
3. В чем преимущество Redis перед реляционными БД?
4. Как Redis хранит данные?
5. Какие структуры данных есть в Redis?

7.2. Кэширование

1. Какие реализации кэша можно использовать со Spring `@Cacheable`?
2. Какие кэши встроены в Spring?
3. Какие кэши требуют внешнего сервиса или дополнительной зависимости?
4. Чем распределенный кэш отличается от локального?
5. Когда подходит локальный кэш, а когда нужен распределенный?
6. Что такое инвалидация кэша и какие проблемы она решает?
7. Какие алгоритмы вытеснения кэша есть в Redis?
8. Чем `allkeys-lru` отличается от `volatile-lru`?
9. Как выбрать eviction policy?

7.3. Персистентность

1. Чем in-memory-хранилище отличается от персистентного?
2. Как Redis сохраняет данные на диск?
3. Что такое RDB snapshot?
4. Что такое AOF log?
5. Чем RDB отличается от AOF?
6. Может ли Redis потерять данные при записи?
7. Как RDB, AOF и replication влияют на риск потери данных?

7.4. Очереди и Pub/Sub

1. Как в Redis можно реализовать очередь?
2. Как Redis Lists используются для очередей?
3. Что такое Redis Pub/Sub?
4. Для каких задач подходит Pub/Sub и почему это не полноценная надежная очередь?
5. Что такое Redis Streams и чем они отличаются от Pub/Sub?
6. Что такое consumer group в Redis Streams?

7.5. Sentinel, Cluster и hash slots

1. Что такое Redis standalone, Sentinel и Cluster?
2. Чем standalone отличается от Sentinel?
3. Чем Sentinel отличается от Cluster?
4. Какие задачи решает Sentinel?
5. Какие задачи решает Cluster?
6. Что такое hash slot и где используются hash slots?

7.6. Distributed lock и транзакции

1. Как реализовать distributed lock в Redis?
2. Почему простой `SETNX` может быть опасен?
3. Что делает команда `SET key value NX PX ?`
4. Что такое Redlock?
5. Как Redis реализует транзакции?
6. Что делают `MULTI`, `EXEC` и `WATCH` ?
7. Есть ли rollback в Redis-транзакциях?
8. Что Redis дает с точки зрения CAP-теоремы?

8. Kafka

8.1. Базовая архитектура

1. Что такое Kafka и для каких задач она используется?
2. Из каких компонентов состоит Kafka-кластер?
3. Что такое broker, topic и partition?
4. Что такое producer и consumer?
5. Что такое consumer group?
6. Что такое offset и где он хранится?

8.2. Партиции, запись и порядок сообщений

1. На что влияет количество partitions?
2. Как происходит запись сообщения в Kafka?
3. Как связаны количество producer, consumer и partitions?
4. Может ли consumer group содержать больше consumer, чем partitions?
5. Что произойдет, если consumer больше, чем partitions?
6. Какие гарантии порядка предоставляет Kafka?
7. В рамках чего Kafka гарантирует порядок сообщений?

8.3. Consumer group и rebalance

1. Что такое rebalance?
2. Когда запускается rebalance?

3. Что происходит во время rebalance?
4. Как rebalance влияет на обработку сообщений?

8.4. Гарантии доставки

1. Какие гарантии доставки предоставляет Kafka?
2. Что такое at-most-once?
3. Что такое at-least-once?
4. Что такое exactly-once?
5. Как настраиваются гарантии доставки в Kafka?
6. Что такое idempotent producer?
7. Чем idempotent producer отличается от обычного producer?

8.5. Репликация и отказоустойчивость

1. Как Kafka переживает падение broker?
2. Что такое replication factor?
3. Что такое leader partition?
4. Что такое follower?
5. Что такое ISR?
6. Как leader и ISR участвуют в подтверждении записи?

8.6. Retention, compaction и transaction log

1. Какие политики очистки данных есть в Kafka?
2. Что такое retention?
3. Что такое log compaction?
4. Как размер сегмента связан с очисткой данных?
5. Что такое transaction log в Kafka?
6. Для чего используется transaction log?

8.7. Транзакции и exactly-once semantics

1. Как устроены транзакции в Kafka?
2. Что такое transactional producer?
3. Что такое transaction coordinator?
4. Что означает exactly-once semantics в Kafka?

8.8. Сжатие, Zero Copy и память

1. Как Kafka работает со сжатыми данными?
2. Какие алгоритмы сжатия поддерживает Kafka?
3. Где происходит сжатие и распаковка сообщений?
4. Что такое Zero Copy?
5. Как Zero Copy ускоряет Kafka?
6. Как Kafka использует page cache?
7. Как Kafka producer работает с памятью?

8. Какие буферы есть у Kafka producer?
9. Как Kafka broker использует heap, off-heap и page cache?
10. Как Kafka consumer использует память?

8.9. Мониторинг Kafka

1. Какие метрики Kafka нужно мониторить?
2. Что такое consumer lag?
3. Какие алерты по Kafka стоит настраивать?
4. Как мониторить broker, producer и consumer?

9. Kubernetes и Istio

9.1. Базовые ресурсы Kubernetes

1. Что такое Pod и из чего он состоит?
2. Может ли в Pod быть несколько контейнеров?
3. Что такое Deployment и для чего он нужен?
4. Как Deployment связан с ReplicaSet?
5. Что такое ReplicaSet и чем он отличается от ReplicationController?
6. Что такое rolling update?

9.2. Labels, selectors и namespaces

1. Для чего нужны labels в Kubernetes?
2. Что такое selector?
3. Как labels и selectors используются в Service и Deployment?
4. Для чего нужны namespaces?
5. Как namespaces помогают разделять ресурсы?

9.3. Service, DNS и сетевое взаимодействие

1. Что такое Service в Kubernetes и зачем он нужен?
2. Какие типы Service существуют?
3. Чем ClusterIP отличается от NodePort и LoadBalancer?
4. Как работает DNS в Kubernetes?
5. Как обратиться к сервису внутри кластера?
6. Как выглядит DNS-имя сервиса?
7. Как Pod получает IP?
8. Как Pod общаются между собой?
9. Как Service балансирует трафик на Pod?
10. Что делает kube-proxy?

9.4. kubectl и диагностика

1. Что такое `kubectl` ?

2. Какие основные команды `kubectl` используются?
3. Чем `kubectl get` отличается от `kubectl describe`?
4. Как посмотреть логи Pod?
5. Как зайти внутрь Pod?
6. Как диагностировать проблему с Pod, Service или Deployment?

9.5. Архитектура Kubernetes

1. Какие компоненты входят в control plane?
2. За что отвечает kube-apiserver?
3. За что отвечает scheduler?
4. За что отвечает controller-manager?
5. За что отвечает etcd?
6. Что делает kubelet на worker node?
7. Что делает kube-proxy?

9.6. RBAC и ServiceAccount

1. Что такое RBAC в Kubernetes?
2. Из каких сущностей состоит RBAC?
3. Чем Role отличается от ClusterRole?
4. Чем RoleBinding отличается от ClusterRoleBinding?
5. Что такое ServiceAccount и где он применяется?

9.7. NetworkPolicy

1. Что такое NetworkPolicy?
2. Для чего нужны сетевые политики?
3. Как ограничить входящий трафик к Pod?
4. Как ограничить исходящий трафик из Pod?

9.8. Istio и service mesh

1. Что такое service mesh?
2. Какие задачи решает service mesh?
3. Какие ресурсы Istio используются чаще всего?
4. Что такое Istio Gateway?
5. Что такое VirtualService?
6. Что такое DestinationRule?
7. Для чего нужен ServiceEntry?
8. Как Istio реализует traffic management?
9. Как Istio помогает с observability?
10. Как Istio помогает с security?